

10. Conclusion

The Boundora Book Store project delivers a practical, locally-tailored online bookstore that supports discovery, ordering, and administration for physical book sellers. The system balances a pragmatic payment model (static QR + Telegram verification) with a modern, modular architecture that can evolve into a richer marketplace over time.

Key Achievements

- 1. Full-Stack Delivery:** User and Admin frontends built with Vue 3 and Pinia, and a NestJS backend providing modular services and APIs.
- 2. Local Payment Flow:** Implemented a static QR payment flow integrated with Telegram for proof submission and verification, matching local banking behaviors.
- 3. Admin Tooling:** Genre and product management, promotions, order review, and payment-proof moderation for operational control.
- 4. Security & Data Protection:** JWT-based auth, password hashing, validation pipes, role-based guards, and secure file handling for payment proofs and cover images.
- 5. Scalable Design:** Stateless services, job queues for background processing, and CDN-friendly static assets to enable scale when needed.
- 6. Deployment Ready:** Dockerized development environment and CI-friendly structure suitable for Render/other cloud hosts.

Technical Accomplishments

Metric	Achievement
Database Entities	Product, Genre, Order, Invoice, User, Role, etc.
Backend Modules	Auth, Products, Orders, Promotions, Telegram
Frontend Applications	User storefront and Admin dashboard (Vue 3)
Auth Methods	Local (email/password), JWT
API Endpoints	Comprehensive product/order/payment endpoints
Automated Jobs	Payment proof processing, image tasks (queued)
Integrations	Telegram bot, Cloudinary/local uploads

Learning Outcomes

- Built a solution adapted to local payment habits (static QR + manual/Telegram verification).
- Designed modular APIs that can be split into microservices later.
- Implemented robust validation and admin moderation workflows for catalogue and payments.

Next Steps

- Add optional inventory and seller management in a future phase to support multi-vendor scenarios.

- Introduce payment webhooks/auto-matching where bank integrations are available.
- Build recommendation and personalization components to boost discovery and conversion.
- Deliver mobile/PWA experience for wider reach and improved checkout UX.

The Boundora Book Store is production-capable for its initial scope and positioned for iterative enhancement based on operational feedback and marketplace needs.

Our Solution: Implement an integrated system for **order management and sales monitoring**.

1.1.4 Transaction Trust and Reliability Challenge

Trust is essential for online book transactions:

- **Customer Concerns:** "Will I receive the correct book?" "Is my payment safe?"
- **Seller Concerns:** "How do I confirm payments?" "How are refunds handled?"

Our Solution:

- Secure online payment processing with transaction verification (from admin side)
- Clear order confirmation and status tracking from admin
- Rating and review system to improve transparency

1.2 Objectives

The primary objectives of the Boundora Book Store project are:

1. **Develop a User-Friendly Online Bookstore:** Create an intuitive web interface where users can browse, search, and purchase books easily.
2. **Improve Order Management:** Ensure reliable order tracking and reduce processing errors; inventory reconciliation is manual initially.
3. **Create an Admin Management Dashboard:** Provide administrators with tools to manage books, users, orders, and categories efficiently.
4. **Integrate Secure Payment Processing:** Enable safe and reliable online transactions with proper payment status handling.
5. **Ensure System Security:** Implement robust authentication (including OAuth 2.0), authorization, and data protection mechanisms to safeguard user information.
6. **Ensure System Security:** Implement authentication, authorization, and data protection mechanisms to safeguard user data.
7. **Organize Books Effectively:** Support categorization by genre, author, and other attributes for better discovery.

1.3 Scope

In Scope

User-Facing Features:

- User registration with username/email/password
- User authentication and profile management
- Category and genre filtering
- Book detail pages with images and descriptions
- Shopping cart and checkout functionality
- Secure online payment processing via telegram
- Booking management (view, cancel)
- Hotel rating and review submission
- Wishlist management for users
- Offer Promotion/New arrivals/ Best selling book for suggestion

Admin Features:

- Dashboard with sales and user analytics
- Promotion, Product and Genre management
- Order management (view, update status)
- Payment and transaction monitoring

System Features:

- Order status lifecycle management
- Image storage for book covers
- Automated order confirmation
- Secure API communication between frontend and backend/auth-service

Out of Scope

The following features are not included in the current version:

- Mobile applications (iOS/Android)
- Multi-language support
- Multi-currency support
- E-book or digital download support
- Subscription or membership system
- Loyalty or reward programs
- Live chat or messaging system
- Third-party marketplace integration

2. System Design

2.1 Project Structure

The project follows a monorepo structure with clearly separated frontend and backend codebases:

```
Book-Store-Project/  
├── docker-compose.yml
```

```

├── README.md
├── auth-service/                                # NestJS Auth-Service API
│   ├── package.json
│   ├── src/
│   │   ├── main.ts
│   │   ├── app.module.ts
│   │   └── entities/
├── authentication and authorization
│   ├── auth/                                # For login and register
│   ├── common/                             # Apply Pipe validation
│   ├── database/                           # Initialize the data
│   └── uploads/
├── backend/                                    # NestJS Backend API
│   ├── package.json
│   ├── src/
│   │   ├── main.ts
│   │   ├── app.module.ts
│   │   ├── config/                         # Database, Cloudinary, Upload configs
│   │   ├── coupon/                        # Giving code for discount
│   │   ├── Database/                      # Main Database
│   │   ├── invoice/                      # Payment Implementation
│   │   ├── newarrival/                   # Adding to New Book as a new arrival
│   │   ├── order/                        # Managing user order
│   │   ├── product/                      # Book storage
│   │   ├── product-genre/                # Count books based on genre
│   │   ├── promotion/                   # Discount
│   │   ├── recommendation/              # Book suggestion
│   │   ├── telegram/                    # Notification management
│   │   └── wishlist/                     # Manage each user wishlist
│   └── uploads/                           # Local file storage (dev)
├── frontend/
│   └── src/
│       ├── assets/                        # Images, icons, global styles
│       ├── components/                   # Reusable UI components
│       │   ├── client/                   # Client-side components
│       │   └── admin/                   # Admin-side components
│       ├── layouts/                      # Application layouts
│       │   ├── client/                   # Client layouts
│       │   └── admin/                   # Admin layouts
│       ├── router/                       # Vue Router configuration
│       │   └── index.js                  # Main router
│       ├── services/                     # API & external services
│       ├── store/                        # State management (Pinia)
│       ├── views/                        # Page-level views
│       │   ├── client/                  # Client pages
│       │   └── admin/                  # Admin pages
│       ├── App.vue                       # Root component
│       └── main.js                       # App entry point

```

2.2 System Architecture

The Hotel Booking System follows a **three-tier architecture** with clear separation of concerns:

Layer	Components	Technologies
Presentation	User Frontend, Admin Frontend	Vue.js 3, Pinia, Vue Router, Axios
Application	Backend API, Auth-Service API	NestJS, Passport, JWT, Validation Pipe
Data	Database, Payments, Storage	PostgreSQL (Neon DB), Dbeaver

Data Flow:

- Login / Register Frontend → (HTTPS/REST) → Auth-Service API
- User Frontend → (HTTPS/REST) → Backend API
- Admin Frontend → (HTTPS/REST) → Backend API
- Backend API → PostgreSQL

2.3 Architecture Patterns

2.3.0 Auth-Service Architecture Pattern: Modular MVC

The NestJS auth-service follows a **Modular MVC (Model-View-Controller)** pattern:

Layer	Responsibility	Implementation
Controllers	Handle HTTP requests, route to services	*.controller.ts files
Services	Business logic, data processing	*.service.ts files
Entities	Data models, database schema	entities/*.entity.ts
DTOs	Request/Response validation	dto/*.dto.ts
Guards	Authorization middleware	guards/*.guard.ts
Strategies	Authentication strategies	strategies/*.strategy.ts

2.3.1 Backend Architecture Pattern: Modular MVC

The NestJS backend follows a **Modular MVC (Model-View-Controller)** pattern:

Layer	Responsibility	Implementation
Controllers	Handle HTTP requests, route to services	*.controller.ts files
Services	Business logic, data processing	*.service.ts files
Entities	Data models, database schema	entities/*.entity.ts

2.3.2 Frontend Architecture Pattern: Component-Based with Composition API

The Vue.js frontends utilize:

Pattern	Purpose	Implementation
Composition API	Logic reuse, better TypeScript support	<code><script setup></code> syntax
Pinia State Management	Centralized state, reactive stores	<code>stores/*.ts</code>
Vue Router	Client-side routing, route guards	<code>router/index.ts</code>

2.3.3 Database Design Pattern: Relational with TypeORM

- **Entity-Relationship Model** with normalized tables
- **TypeORM Repository Pattern** for data access
- **Query Builder** for complex queries
- **Auto-synchronization** for development (disabled in production)

2.4 Entity Relationship Diagram (ERD)

Entity Relationships:

Parent Entity	Relationship	Child Entity
USER	has many	USER_ROLE
ROLE	has many	USER_ROLE
USER	has many	REFRESH_TOKEN
USER	places many	ORDER
ORDER	contains many	ORDER_ITEM
ORDER	applies one	COUPON
PRODUCT	belongs to many	GENRE
GENRE	contains many	PRODUCT
PRODUCT	appears in many	ORDER_ITEM
PRODUCT	belongs to many	PROMOTION
PROMOTION	applies to many	PRODUCT
PRODUCT	appears in many	NEW_ARRIVAL
PRODUCT	appears in many	RECOMMENDATION
USER	has many	WISHLIST
USER	links to one	TELEGRAM_LINK

Key Entity Attributes:

Entity	Key Attributes
USER	id, email, username, passwordHash, isActive createdAt

Entity	Key Attributes
ROLE	id, name
USER_ROLE	id, userId, roleId
REFRESH_TOKEN	id, userId, tokenHash, expiresAt, revokedAt, createdAt
PRODUCT	id, title, author, description, image, price, countSold, published, pages, language, rating
GENRE	id, name, description, image, svgIcon
PRODUCT_GENRE	product_id, genre_id
ORDER	id, customerName, customerEmail, customerPhone, customerAddress, subtotal, discountAmount, couponCode, totalAmount, status, paymentMethod, paymentImage, bankName, telegramChatId, userId, invoicePath, invoicePreviewPath, createdAt, updatedAt
ORDER_ITEM	id, orderId, productId, quantity, unitPrice, totalPrice
COUPON	id, code, name, discountType, discountValue, minimumPurchase, maximumDiscount, usageLimit, usageCount, usageLimitPerUser, startDate, endDate, isActive, isFirstTimeOnly, description
PROMOTION	id, name, type, discount, startDate, endDate, status, description, badgeText, createdAt
NEW_ARRIVAL	id, productId, isActive, priority, expiresAt, createdAt
RECOMMENDATION	id, productId, isActive, priority, expiresAt, createdAt
WISHLIST	id, userId, productId, createdAt
TELEGRAM_LINK	id, userId, chatId, createdAt

2.5 API Architecture

Module	Base URL	Description
Auth	/auth	Authentication, registration, login, JWT-based user info, candidate list
Coupons	/coupons	Create, update, delete, validate, and use discount coupons
Genres	/genres	CRUD operations for product genres, including image upload
New Arrivals	/new-arrivals	Manage new arrival products, CRUD and list active items
Orders	/orders	Create, view, update order status, upload payment images, delete orders
Products	/products	CRUD operations for products, including image upload
Product Genres	/product_genres	List all product-genre relationships, count products per genre

Module	Base URL	Description
Promotions	/promotions	Manage promotions and discounts, CRUD operations
Recommendations	/recommendations	Manage product recommendations, CRUD operations
Telegram	/telegram	Link users to Telegram, check connection status, optional webhook
Wishlists	/wishlists	Manage user wishlists, add/remove products, list wishlist items

3. Technology Stack

3.1 Backend Technologies

Technology	Version	Purpose
NestJS	11.x	Backend framework
TypeScript	5.7.x	Type-safe development
TypeORM	0.3.28	ORM for database
PostgreSQL (pg)	8.16 / 8.18	Primary database client
Multer	2.0.2	File upload handling
Class-validator	0.14.3	Validation decorators
Class-transformer	0.5.1	Transform DTOs / objects
RxJS	7.8.1	Reactive programming
Reflect-metadata	0.2.2	Metadata reflection
Sharp	0.33.2	Image processing
Jest	30.0.0	Testing framework

3.2 Auth-Service Technologies

Technology	Version	Purpose
NestJS	11.x	Backend framework
TypeScript	5.7.3	Type-safe development
TypeORM	0.3.28	ORM for database
PostgreSQL (pg)	8.18.0	Primary database client
Passport/JWT	11.x / 4.0.1	Authentication
BcryptJS	3.0.3	Password hashing
Class-validator	0.14.3	Validation decorators

Technology	Version	Purpose
Class-transformer	0.5.1	Transform DTOs / objects
RxJS	7.8.1	Reactive programming
Multer	2.0.2	File upload handling
Reflect-metadata	0.2.2	Metadata reflection
Jest	30.0.0	Testing framework

3.3 Frontend Technologies

Technology	Version	Purpose
Vue.js	3.5.26	Frontend framework
Vue Router	4.6.4	Client-side routing
Pinia	3.0.4	State management
Axios	1.13.2	HTTP client
FontAwesome Vue	3.1.3	Icon library
Vite	7.3.0	Build tool / dev server
Tailwind CSS	3.4.17	Utility-first CSS framework
@tailwindcss/forms	0.5.9	Tailwind form styles
PostCSS	8.4.43	CSS transformations
Autoprefixer	10.4.20	CSS vendor prefixing
ESLint	9.39.2	Linting
Prettier	3.7.4	Code formatting
Vitest	4.0.16	Unit testing
@vue/test-utils	2.4.6	Vue component testing
Vite Plugin Vue Devtools	8.0.5	Vue devtools integration
jsdom	27.3.0	DOM emulation for tests
esbuild	0.27.3	Bundling / build optimization

3.3 DevOps & Deployment

Technology	Purpose
Render	Backend & Frontend hosting
Neon	PostgreSQL database hosting

Technology	Purpose
GitHub Actions	CI/CD auto-deploy
Docker/Docker Compose	Local development

4. Features

4.1 User Features

4.1.1 Authentication & Authorization

Feature	Description
User Registration	Create account with username/email/password with pipe validation
Username/Password Login	Secure login using bcrypt password hashing
JWT Session Management	Stateless authentication with 7-day token expiration
Profile Management	Update personal information and profile picture
Account Deactivation Handling	Graceful handling when admin deactivates user account

4.1.2 Book Discovery & Browsing

Feature	Description
Book Listing	Browse all available books with images, titles, and authors
Book Details	View detailed info: description, genre, price, and ratings
Search & Filter	Search books by title, author, or genre; filter by price or availability
New Arrivals	Highlight newly added books
Promotions & Discounts	View discounted books or promotional offers
Recommendation	Highlight newly recommended books
Best Selling Products	Display best selling product based on countSold > 10

4.1.3 Shopping Cart & Wishlist

Feature	Description
Add to Cart	Add single or multiple books to shopping cart
Update Cart	Adjust quantities or remove books from cart
Wishlist	Save favorite books for later purchase
Cart Summary	View total price with discount and tax calculation
Coupon Code Application	Apply valid coupon codes for discounts

4.1.4 Order & Payment System

Feature	Description
Order Placement	Submit order with customer info and selected books
Payment Integration	Pay securely via KHQR or bank transfer
Payment Status Tracking	Monitor payment status: Pending, Paid, Failed
Payment Upload	Upload payment proof image if required
Order Cancellation	Cancel orders if not yet paid (remove to trash)

4.1.5 Telegram Integration

Feature	Description
Telegram Linking	Connect account with Telegram for order updates
Order Notifications	Receive order status updates via Telegram messages
Webhook Support	Optional webhook endpoint for Telegram messages

4.2 Admin Features

4.2.1 Dashboard & Analytics

Feature	Description
Overview Dashboard	Central view of system metrics: total books, genres, promotion, subscription
Sales Statistics	Charts showing sales trends, daily/monthly revenue
User Statistics	Monitor user registrations, active users, and engagement
Chart.js Visualizations	Interactive charts for sales, user activity, and top-selling books

4.2.2 Book Management

Feature	Description
Book List	View all books with search and filter options
Create Book	Add new books with title, author, genre, price, page, language, published date
Edit Book	Update book details (title, author,image, etc)
Delete Book	Remove books from the catalog
Image Management	Upload, reorder, or delete book cover images
Stock Management	(Not implemented) Admin reconciles stock manually
Discount & Promotion	Apply percentage discounts or promotional offers or Hot badge or Buy 1 Free 1

4.2.3 Order Management

Feature	Description
Order List	View all orders with filtering by status
Order Details	See full order info: items, total price, and payment status
Approve Orders	Confirm pending orders and start payment timer if required
Reject Orders	Cancel orders with reason (e.g., invalid product or other reasons)
Cancel Orders	Cancel confirmed orders if needed

4.2.4 Payment & Transaction Monitoring

Feature	Description
Payment History	View all paid and pending payment transactions
Payment Status	Track payment statuses: Pending, Paid, Failed
Transaction Details	View Stripe transaction IDs or payment proof images
Revenue Tracking	Monitor total revenue and refunds

4.2.5 Telegram & Notification Management

Feature	Description
Telegram Notifications	Configure automated notifications for orders or promotions
Webhook Management	Manage webhook endpoints for Telegram or other messaging
Broadcast Messages	Send messages to all linked users via Telegram

4.2.6 Genre Management

Feature	Description
Create Genre	Add new genres with name, description, image, slug, and SEO metadata
Edit Genre	Update genre details, change associated image or slug
Delete Genre	Soft-delete or hard-delete genres (with reassignment options for products)
Bulk Import/Export	CSV import/export for genres to speed up catalog setup
Merge / Split Genres	Merge duplicate genres or split a genre into multiple refined categories
Assign/Unassign	Assign multiple genres to books and remove associations in bulk
Visibility Toggle	Enable/disable genres (show/hide on frontend)
Priority & Ordering	Set display priority, sort order and featured genres for homepage widgets

Feature	Description
Slug & Synonyms	Auto-generate SEO-friendly slugs and manage synonyms/aliases for search
Analytics per Genre	View counts: number of books, sales by genre, and conversion metrics
Image & Icon Handling	Upload and crop genre cover images, support svg and webp with validation

Admin UI notes:

- Provide confirmation and reassignment options when deleting a genre to avoid orphaned products.
- Use client-side validation for fast feedback, server-side validation for security and integrity.
- Expose bulk operations with progress indicators and rollback support on failure.

5. Security Implementation

5.1 Authentication Security

Measure	Implementation
Password Hashing	bcrypt with 10 salt rounds
Password Requirements	Pipe validation
JWT Token	HMAC-SHA256, 7-day expiration
Stateless Auth	No server-side sessions

5.2 Authorization

Guard	Purpose
JwtAuthGuard	Validates JWT on protected routes
@Private()	Marks route based on user role

5.3 Additional Security

Security Layer	Implementation
HTTP Headers	Helmet.js (X-Frame-Options, XSS Protection, HSTS, etc.)
CORS	Configurable origins, credentials enabled
Input Validation	ValidationPipe with auth, classic-transform/classic-validator
SQL Injection	TypeORM parameterized queries
File Upload	Allowed formats validation (jpg, jpeg, png, webp)

6. Business Logic

6.1 Order Status Lifecycle

The bookstore order system tracks orders from creation to completion:

Current State	Action/Trigger	Next State	Notes
(Start)	User places order	PENDING	Order recorded
PENDING	Admin confirms	CONFIRMED	Reservation noted for payment
PENDING	Admin rejects	REJECTED	Order rejected
PENDING	User cancels	REJECTED	Order cancelled
CONFIRMED	Payment completed	PAID	Order processed and fulfilled
CONFIRMED	Payment timeout	REJECTED	Order rejected

Status Details:

Status	Description	Inventory Impact	Next Actions
PENDING	Order placed, awaiting admin review	N/A	Admin: Confirm/Reject; User: Cancel
CONFIRMED	Order approved, awaiting payment	N/A	User: Pay; System: Auto-reject if timeout
PAID	Payment received, order fulfilled	N/A	User: Track shipment
REJECTED	Order rejected or cancelled	N/A	None

6.2 Inventory Management (Not Implemented)

This project does not include a built inventory module yet. Stock is managed manually by administrators outside the system for the current release.

Notes:

- Orders are recorded but do not reserve stock automatically.
- Admins should reconcile stock and update product availability manually when required.
- When inventory management is added in a future release, it should follow transactional patterns and short-lived reservations to prevent oversell.

6.3 Price Calculation

Price calculation for bookstore orders is deterministic and follows these steps (no tax applied):

1. **Line Item Price:** For each product line: `linePrice = product.basePrice × quantity`.
2. **Product-Level Discounts:** Apply product-specific discounts or promotions: `linePrice = linePrice × (1 - productDiscount/100)`.
3. **Bundle/Promotion Rules:** Apply cart-level promotions (e.g., "buy 2 get 1", bundle discounts) according to priority rules; ensure promotions are applied in a deterministic order to avoid conflicts.
4. **Coupon Application:** If a coupon code is provided, validate coupon constraints (minimum purchase, user eligibility, expiry) and apply percentage or fixed discounts.

5. **Shipping & Fees:** Calculate shipping fee based on shipping method (flat, weight, or threshold-free shipping). Add handling fees if applicable.
6. **Final Total:** $\text{total} = \text{subtotal} - \text{discounts} + \text{shipping}$.

Example: Book A \$20 (2 qty), Book B \$15 (1 qty), product discount 10% on Book A, coupon \$5 off, shipping \$3.

- Line totals: Book A: $\$20 \times 2 = \$40 \rightarrow \text{after } 10\% = \36 ; Book B: $\$15 \times 1 = \15 ; Subtotal = \$51
- Apply coupon: $\$51 - \$5 = \$46$
- Shipping: $\$46 + \$3 = \$49$
- Final total = \$49

Implementation notes:

- Calculate amounts in the smallest currency unit (e.g., cents) to avoid floating-point errors.
- Apply validation to ensure discounts do not produce negative totals.
- Record a full pricing breakdown on the order for auditing and refund calculations.

6.4 Payment System

The bookstore uses a static QR-code payment model integrated with Telegram notifications. There is no automatic payment expiration window — payments are confirmed when proof is received and verified.

Behavior and flow:

1. **Static QR Code:** The system displays a static, pre-configured QR code (KHQR or bank QR) for the store's account on the checkout page and in the order confirmation view. The QR is not per-order — it is a store-level code the customer scans to pay.
2. **User Payment:** After the customer scans and completes payment with their banking app, they upload payment proof (screenshot) via the checkout flow or send proof through Telegram if linked.
3. **Telegram Notification:** If the user links their Telegram account, the system can accept payment proof via Telegram messages. The Telegram bot forwards image/text to the backend and associates it with the order using provided order ID or reference.
4. **Verification:** Admins (or an automated matcher) verify payment proof against bank statements or transaction references. The system supports:
 - **Manual verification:** Admin reviews uploaded screenshot and marks order **PAID**.
 - **Auto-matching (optional):** If the bank/processor supplies transaction IDs or webhooks, the backend can automatically match payments to orders and mark **PAID**.
5. **Order Status Transition:** When payment is verified, order status transitions to **PAID**. Until verification, orders remain **CONFIRMED** (if admin already approved) or **PENDING**.
6. **No Expiration:** There is no forced auto-cancellation due to payment timeout. Admins may manually reject orders after a configurable internal policy timeframe, but the system will not automatically change status solely based on elapsed time.

Implementation notes:

- **Proof Storage:** Save uploaded payment images securely (Cloudinary/local) and store metadata (uploader, timestamp, order reference) for audit.
- **Telegram Integration:** Use a Telegram bot webhook to receive messages and media; associate incoming messages by parsing order references or by prompting the user for an order ID.

- **Idempotency & Safety:** Ensure marking **PAID** is idempotent and records who verified the payment and when.
- **Admin Tools:** Provide admin UI to list unverified payment proofs, filter by order/date, accept/reject proofs, and add verification notes.
- **Optional Bank Webhooks:** If available, implement webhook endpoints to receive bank/processor confirmations for automatic matching and reconciliation.

6.6 Purchasing Validation Rules

Validation	Rule	Error Message / Behavior
Product Availability	Product must be active and have required data (SKU/ISBN)	"Product not available"
Coupon Validity	Coupon exists, not expired, meets min purchase, usage limits	"Coupon is invalid or not applicable"
Address Validation	Shipping address must contain required fields (name, phone, addr)	"Invalid shipping address"
Payment Method	Selected payment method supported and has necessary credentials	"Payment method unavailable"
Max Quantity	Enforce product-level max per-order limits	"Exceeds maximum allowed quantity for this product"
Duplicate Order Check	Prevent accidental double orders (same cart/payment within X mins)	Reject duplicate or flag for manual review
Product Data Integrity	Product must have price, SKU/ISBN, and active status	"Product data incomplete; contact admin"

These validation rules run both on the client for early feedback and on the backend for enforcement.

7. Deployment

7.1 Deployment Architecture

Component	Platform	Type
User Frontend	Render	Static Site
Admin Frontend	Render	Static Site
Backend API	Render	Web Service
Auth-Service API	Render	Web Service
PostgreSQL	Neon DB	Database

CI/CD: GitHub push → Render auto-build → Deploy

7.2 Deployment Configuration

Backend (Render Web Service):

Setting	Value
Root Directory	backend
Build Command	npm install && npm run build
Start Command	npm run start:prod
Auto-Deploy	Yes (production branch)

Auth-Service (Render Web Service):

Setting	Value
Root Directory	auth-service
Build Command	npm install && npm run build
Start Command	npm run start:prod
Auto-Deploy	Yes (production branch)

Frontend (Render Static Sites):

Setting	Value
Root Directory	frontend
Build Command	npm install && npm run build
Publish Directory	dist
Rewrite Rule	/* → /index.html → Overwrite

7.3 URLs

Service	URL
User Frontend	https://boundora-0f1i.onrender.com
Admin Frontend	https://boundora-0f1i.onrender.com
Backend API	https://bookstore-project-4ugp.onrender.com
Auth-Service API	https://auth-service-5o4z.onrender.com

8. Challenges & Solutions

8.1 Validation Handling

Challenge: Implementing comprehensive input validation across the entire application to prevent invalid data from entering the system while providing meaningful error messages.

Why Both Backend AND Frontend Validation?

Layer	Purpose	Benefits
Frontend	User experience, instant feedback	Prevents unnecessary API calls, shows errors before submission
Backend	Security, data integrity	Protects against malicious requests, API consumers, bypassed frontend

Solution: Dual-layer validation:

- **Frontend (Vue.js):** Provides instant feedback with regex validation for emails, password length checks, and form completeness validation before submission
- **Backend (NestJS DTO):** Uses class-validator decorators for security-critical validation including email format, password requirements (8+ characters, uppercase, lowercase, number), and sanitization

9. Future Plans

9.1 Microservice Architecture

Service	Responsibilities
Auth Service	User Auth, JWT/OAuth, Profiles
Booking Service	Bookings, Availability, Scheduling
Payment Service	Stripe, KHQR, Refunds
Hotel Service	Hotels, Rooms, Amenities
Notification Service	Email, SMS, Push

9.2 KHQR Payment Integration

Cambodian QR payment standard for local payment support:

1. User selects KHQR Payment
2. Backend requests QR generation from Bank
3. Frontend displays QR Code
4. User scans and pays via Banking App
5. Bank sends webhook to Backend
6. Booking confirmed

10. Conclusion

The Hotel Booking Website project successfully delivers a comprehensive, modern hotel booking solution that addresses the core challenges faced by the hospitality industry. Through careful system design and

implementation, the team has created a platform that serves both guests seeking accommodation and administrators managing hotel operations.

Key Achievements

- 1. Full-Stack Implementation:** Complete end-to-end solution with Vue.js frontend applications and NestJS backend API, demonstrating proficiency in modern web development technologies.
- 2. Robust Business Logic:** Implementation of complex hotel booking workflows including real-time availability checking, booking status lifecycle management, and automated payment expiration handling.
- 3. Security-First Approach:** Multiple layers of security including JWT authentication, bcrypt password hashing, Google OAuth 2.0, role-based access control, and comprehensive input validation.
- 4. Third-Party Integration:** Successful integration with Stripe for payment processing, Google for social authentication, and Cloudinary for cloud image storage.
- 5. Scalable Architecture:** Modular design with clear separation of concerns, preparing the system for future microservice migration and feature expansion.
- 6. Production-Ready Deployment:** Full deployment pipeline with Docker containerization for local development and cloud hosting on Render for backend and frontends, with PostgreSQL database on Neon.

Technical Accomplishments

Metric	Achievement
Database Entities	10+ entities with complex relationships
Backend Modules	9 modules with complete CRUD operations
Frontend Applications	2 apps (User and Admin)
Authentication Methods	3 methods (Local, JWT, Google OAuth)
API Endpoints	50+ RESTful endpoints
Automated Jobs	Payment expiration cron scheduling
Real-Time Features	Availability calculation algorithm
Cloud Integration	Image CDN with transformations

Learning Outcomes

Through this project, the team gained valuable experience in:

- Enterprise-level application architecture
- TypeScript and modern JavaScript frameworks
- Database design and ORM implementation
- RESTful API design and documentation
- Authentication and authorization patterns

- Payment gateway integration
- Cloud deployment and DevOps practices
- Agile development and teamwork

Appendix

A. Default Admin Credentials

The system automatically seeds a default administrator account on first startup:

Field	Value
Email	Rithy@gmail.com
Password	@Heng123
Role	Admin

B. Status Code Reference

Booking Status

Status	Code	Description
Pending	pending	Awaiting admin approval
Confirmed	confirmed	Approved, awaiting payment
Paid	paid	Payment received, order fulfilled
Rejected	rejected	Rejected or cancelled orders

Payment Status

Status	Code	Description
Pending	pending	Payment initiated
Paid	paid	Payment successful
Failed	failed	Payment failed
Refunded	refunded	Payment refunded

End of Final Project Report